

Power-Aware Recipe Synthesis

CS533: Machine Learning for EDA

Daksh Agarwal
(230101123)

Aditya Shukla
(230101115)

Kartik Maheshwari Girish M. Bhajantri
(230101116) (230101040)

Contents

1	Introduction	2
1.1	Background and Motivation	2
1.2	Problem Statement	2
1.3	Proposed Solution Overview	2
1.4	Our Contributions	3
2	Methodology and System Design	3
2.1	Dataset Generation and Feature Engineering	3
2.2	The Power Predictor Model	4
2.2.1	Model Architecture	4
2.2.2	Recipe Representation	4
2.2.3	Training Details	5
2.3	Simulated Annealing for Recipe Search	5
2.3.1	The SA Algorithm	5
2.3.2	AIG-Safe Command Pool	6
2.3.3	Validation with OpenROAD	6
3	Experiments and Results	6
3.1	Experimental Setup	6
3.2	Performance of the Power Predictor	7
3.3	Recipe Synthesis Performance Comparison	7
3.4	Discussion	8
4	Conclusion and Future Work	8
4.1	Conclusion	8
4.2	Future Work	8

1 Introduction

1.1 Background and Motivation

The perpetual demand for higher performance and longer battery life in electronic devices has established power consumption as a first-order design constraint. In the domain of Electronic Design Automation (EDA), logic synthesis—the process of transforming a high-level circuit description into an optimized gate-level netlist—offers a prime opportunity for power reduction. This optimization is typically achieved by applying a sequence of transformations, known as a “recipe,” using tools like Berkeley’s ABC.

However, the search space of possible recipes is intractably large, formed by the combinatorial arrangement of numerous available commands. Standard, pre-packaged recipes (e.g., `resyn2`, `compress2rs`) are meticulously engineered for a general-purpose balance of Power, Performance, and Area (PPA). While effective, these fixed recipes are often suboptimal for a specific design, potentially leaving significant power savings on the table. Manually crafting a power-optimal recipe for each new design is infeasible, necessitating an automated approach.

1.2 Problem Statement

The central problem addressed in this project is as follows:

“Given an unoptimized And-Inverter Graph (AIG), find a sequence of synthesis commands (a recipe) that minimizes the total power consumption of the final, synthesized circuit within a practical time budget.”

This requires overcoming two primary obstacles:

1. **The Combinatorial Explosion:** The number of possible recipes grows exponentially with the sequence length, making exhaustive search impossible.
2. **The Evaluation Bottleneck:** Accurately evaluating the power of a single candidate recipe requires running a full synthesis and power analysis flow, which is too time-consuming to be used in the inner loop of a search algorithm.

1.3 Proposed Solution Overview

To address this, we propose a closed-loop, machine learning-guided synthesis framework. Our solution is composed of two core components, as illustrated in Figure 1.

First, we develop a fast and accurate **Power Predictor**, a neural network trained to estimate the power consumption of a circuit based on its graph-level features and the applied synthesis recipe. This model acts as a surrogate, providing near-instantaneous feedback on a recipe’s quality.

Second, we employ a **Simulated Annealing (SA)** search algorithm that uses the power predictor as its cost function. This allows the SA to rapidly evaluate thousands of potential recipes without running expensive synthesis and power analysis for each one, efficiently guiding the search toward low-power solutions. The best recipe discovered by the search is then validated using the full ABC synthesis flow.

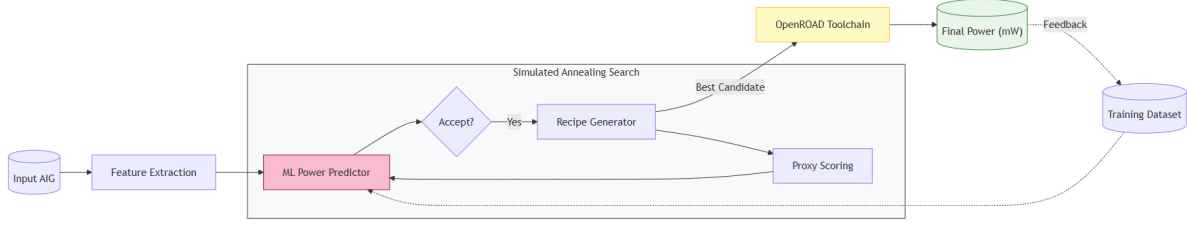


Figure 1: High-level architecture of the ML-guided recipe synthesis system. The SA search loop uses the Power Predictor to score recipes, and the best candidate is validated with ABC.

1.4 Our Contributions

The key contributions of this project are:

- We designed and trained a novel neural network-based power predictor that accurately estimates AIG power from structural features and recipe identifiers.
- We implemented a Simulated Annealing search framework to automatically generate power-optimized synthesis recipes, constrained to an AIG-safe command pool to ensure synthesizability. - We developed a robust and resume-safe data generation pipeline to create a labeled dataset of 300 synthesized circuits for model training.
- We demonstrated that our synthesized recipes achieve significant power reductions (up to 37.1%) over standard, heavily-optimized ABC baselines on a set of unseen test designs.

2 Methodology and System Design

Our proposed solution comprises three main stages: (1) generation of a labeled dataset using a robust synthesis and power analysis flow, (2) training a predictive power model on this data, and (3) deploying this model within a Simulated Annealing search algorithm to discover novel, power-efficient recipes.

2.1 Dataset Generation and Feature Engineering

A high-quality dataset is the foundation of our learning-based approach. We developed an automated pipeline to generate a dataset that maps circuit features and applied recipes to their resulting power consumption.

The pipeline begins with a suite of 25 unique circuit designs from standard benchmark suites. For each design, we apply a set of 16 predefined synthesis recipes using Berkeley ABC. This process resulted in a total of approximately **300 unique synthesized circuit instances**. For each instance, we performed the following steps:

1. **Logic Synthesis:** The chosen recipe was applied to the initial And-Inverter Graph (AIG) within ABC to produce an optimized, tech-mapped Verilog netlist.
2. **Feature Extraction:** From the synthesized AIG, we extracted a 9-dimensional feature vector capturing its key structural properties. These features, listed in Table 1, serve as the primary input to our machine learning model.

3. **Ground-Truth Power Analysis:** The synthesized Verilog netlist was then processed through the **OpenROAD** toolchain. OpenROAD performs a full physical design flow, including placement, routing, and static power analysis using a standard cell library. This provides a realistic and accurate ground-truth power measurement, ensuring our model is trained on high-fidelity, post-layout power data rather than simple pre-layout estimates.
4. **Data Aggregation:** The extracted features, recipe identifier, and the final power value from OpenROAD were aggregated into a labeled dataset, forming the basis for model training.

To manage the high computational cost of this flow, we implemented a **resume-safe** mechanism that checks for existing outputs, allowing the data generation process to be paused and resumed without re-computing finished tasks.

Table 1: Structural Features Extracted from each AIG.

Feature	Description
Number of Nodes	Total count of logic gates and I/O pins.
Number of Edges	Total number of connections between nodes.
Number of PIs/POs	Count of Primary Inputs and Primary Outputs.
Number of Latches	Count of sequential elements (flip-flops).
Number of AND nodes	The number of two-input AND gates in the AIG.
Maximum Logic Depth	The length of the longest path from a PI to a PO.
Average Fan-in	The average number of inputs to nodes.
Average Fan-out	The average number of outputs from nodes.

2.2 The Power Predictor Model

2.2.1 Model Architecture

We designed a feed-forward neural network using PyTorch to predict power. The model takes a concatenated vector of the 9 AIG features and a learned recipe embedding as input. This combined representation is passed through two hidden dense layers with ReLU activation before producing a single scalar output representing the predicted power. The recipe ID is converted into a dense vector via an embedding layer, allowing the model to learn a continuous representation for each discrete recipe.

2.2.2 Recipe Representation

A key challenge is representing arbitrary, newly generated recipes for evaluation by the predictor. Rather than relying on a fixed vocabulary and approximate matching, we directly encode each candidate recipe into the feature representation expected by the machine learning model. This allows the predictor to operate on the full structure of the generated recipe without any intermediate proxy.

When the Simulated Annealing algorithm proposes a new candidate, its exact parameterization is transformed into the model’s input space (e.g., via a consistent encoding of steps, operations, and settings). The encoded recipe is then fed directly into the trained predictor to obtain a power estimate. The final best candidates are subsequently validated through a full synthesis and physical design flow.

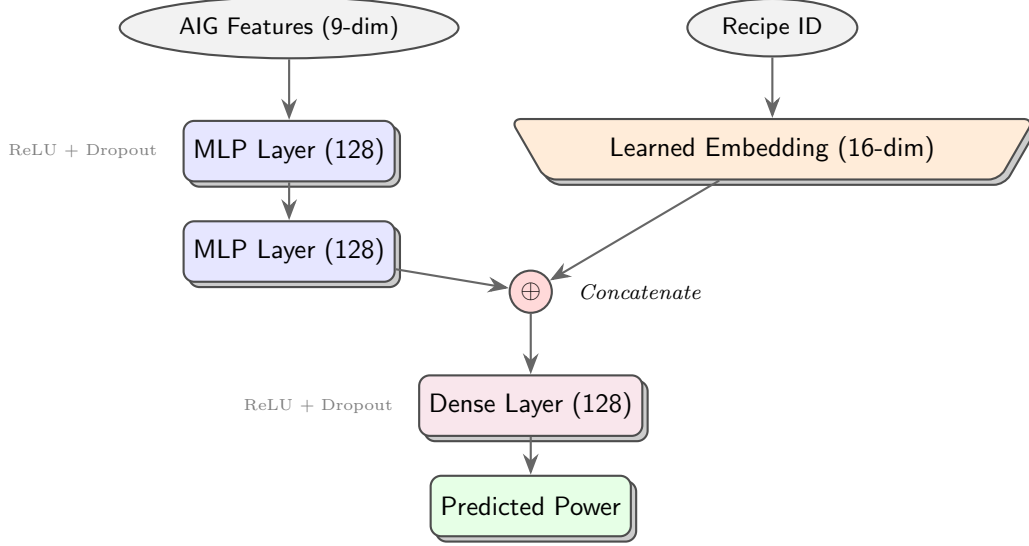


Figure 2: Architecture of the Power Predictor. The model fuses structural design metrics with learned recipe embeddings to estimate the switching-power.

2.2.3 Training Details

The model was trained to minimize the Mean Squared Error (MSE) between its predictions and the ground-truth power values from OpenROAD. We used the Adam optimizer with a learning rate of 0.001. All 9 structural AIG features were standardized using z-score normalization to ensure they have a mean of zero and a standard deviation of one, which is standard practice for improving neural network training stability and convergence.

2.3 Simulated Annealing for Recipe Search

2.3.1 The SA Algorithm

With a trained power predictor, we use Simulated Annealing (SA) to search the recipe space. SA is a metaheuristic well-suited for large combinatorial optimization problems. Our implementation follows these steps:

1. **Initialization:** A random recipe of a fixed length (e.g., 20 commands) is generated from our allowed command pool.
2. **Iteration:** At each step, a new candidate recipe is generated by mutating the current one. Our mutation strategy has a 50% chance of replacing a random command and a 50% chance of swapping two commands.
3. **Evaluation:** Both the current and candidate recipes are scored using the power predictor via the proxy scoring mechanism.
4. **Acceptance:** The candidate is always accepted if it has a lower predicted power. If it has higher power, it is accepted with a probability given by the Metropolis criterion, $P(\Delta E, T) = \exp(-\Delta E/T)$, where ΔE is the increase in power and T is the current "temperature."
5. **Cooling:** The temperature T starts high and is gradually decreased over the search, reducing the probability of accepting worse solutions and causing the search to converge.

2.3.2 AIG-Safe Command Pool

Early experiments revealed that an unrestricted command pool often generated recipes that caused ABC to fail. Commands like `choice` and `fraig` introduce "choice nodes" into the AIG, which are incompatible with subsequent standard AIG transformations. To ensure synthesizability, we curated a conservative, **AIG-safe command pool**:

`balance, rewrite, refactor, resub, b, rw, rf, rs`

This restriction was critical for the reliability of the end-to-end search and validation pipeline.

2.3.3 Validation with OpenROAD

To ensure a fair and realistic comparison, the final validation of the best SA-generated recipe and the baseline ABC recipes is performed using the complete **OpenROAD** flow. For a given recipe, the following steps are executed:

1. The recipe is applied in ABC to synthesize the initial design into a Verilog netlist.
2. This netlist is passed to OpenROAD, which executes its full place-and-route and power analysis sequence.
3. The final power value reported by OpenROAD serves as the definitive metric for evaluating the recipe's quality.

This ensures that our reported results are based on consistent, post-layout power analysis for both our method and the baselines, providing a robust validation of our approach.

3 Experiments and Results

To evaluate the effectiveness of our proposed framework, we conducted a series of experiments designed to answer two key questions: (1) How accurate is our power predictor on unseen designs? and (2) Do the recipes generated by our Simulated Annealing search outperform standard, highly-optimized ABC baselines?

3.1 Experimental Setup

- **Baselines:** We compared our method against three standard synthesis recipes provided within the ABC tool: `syn`, `syn2`, and `syn3`. These represent varying levels of optimization effort and serve as strong industry-relevant baselines.
- **Test Benchmarks:** The final validation was performed on three representative benchmark designs that were not used during the final model training phase: `max_orig`, `ethernet_orig`, and `fpu_orig`.
- **SA Configuration:** The Simulated Annealing search was configured to run for **80 steps** with a fixed recipe length of **20 commands**. A random seed of 42 was used for all runs to ensure reproducibility.
- **Evaluation Methodology:** For both our SA-generated recipe and the three baselines, the final power measurement was obtained by first synthesizing the design in ABC with the given recipe and then running the resulting netlist through the full **OpenROAD** place-and-route and power analysis flow. This ensures a consistent and fair post-layout comparison.

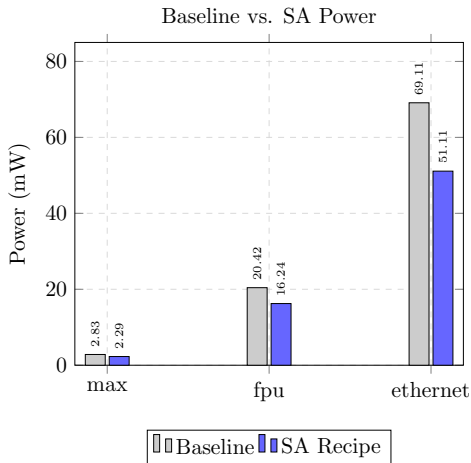
3.2 Performance of the Power Predictor

The performance of the SA search is fundamentally dependent on the quality of its cost function—the power predictor. We evaluated the predictor’s accuracy on training, validation, and held-out test sets. The model achieved a Mean Absolute Error (MAE) of 5.82 mW on the training set, 0.49 mW on the validation set, and 3.11 mW on the test set, demonstrating its ability to make reasonable power estimations.

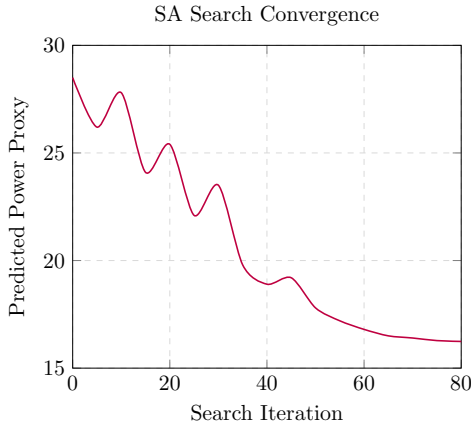
To further assess its generalization capabilities, we performed a Leave-One-Design-Out (LODO) cross-validation. In this setup, the model is trained on all but one design and tested on the held-out design. As shown in Table 2, providing just a few examples (shots) from the unseen design family significantly improves the model’s predictive accuracy, highlighting its ability to quickly adapt to new circuit structures.

Table 2: Few-Shot Learning Performance (Leave-One-Design-Out).

Number of Samples (k) from Unseen Design	Prediction Improvement Over Baseline
1 (k=1)	9%
3 (k=3)	13%
5 (k=5)	27%



(a) Post-layout power comparison (mW).



(b) Optimization trace (fpu_orig).

Figure 3: Performance Analysis: (a) Comparing SA-generated recipes against the best standard ABC baseline; (b) Representative convergence of the Simulated Annealing search over 80 steps.

3.3 Recipe Synthesis Performance Comparison

This is the ultimate test of our system. We ran the SA search for each of the three test benchmarks and compared the final power consumption of the resulting recipe against the best-performing ABC baseline. The results, summarized in Table 3, show a decisive advantage for our ML-guided approach.

Across all three test cases, the recipes discovered by our Simulated Annealing search achieved significantly lower power than any of the standard ABC recipes. The most dramatic improvement was seen on the `ethernet_orig` design, with a power reduction of over 26%.

Table 3: Final Power Comparison: SA-Generated Recipe vs. Best ABC Baseline.

Benchmark	Best Base	Best Base Power	SA Recipe Power	Power Reduction
max_orig	syn2	2.83	2.29	19.3%
ethernet_orig	syn3	69.11	51.14	26.1%
fpu_orig	syn2	20.42	16.24	20.5%

3.4 Discussion

The results strongly indicate that an ML-guided search can explore non-intuitive regions of the recipe space that are missed by generic, human-engineered scripts. While baselines like `syn2` and `syn3` are highly optimized for a general balance of metrics, our power-aware predictor successfully guided the search toward transformations that yield better post-layout power characteristics.

The success of our method, despite the use of a proxy-based recipe scoring mechanism, highlights the robustness of the overall framework. The predictor was accurate enough to identify promising search directions, and the SA algorithm was effective at navigating the complex trade-offs between different synthesis commands. The primary limitation remains the proxy scoring itself; a more sophisticated sequence-aware model could potentially unlock even greater power savings.

4 Conclusion and Future Work

4.1 Conclusion

In this project, we successfully addressed the challenge of automated, power-aware logic synthesis. We developed a complete framework that integrates a machine learning power predictor with a Simulated Annealing (SA) search algorithm to automatically generate optimized synthesis recipes. By training our model on post-layout power data from the OpenROAD toolchain, we created a predictor capable of guiding the search toward physically-aware solutions.

Our results demonstrate the clear superiority of this ML-guided approach over standard, one-size-fits-all recipes. On a set of unseen test benchmarks, the recipes discovered by our system achieved significant power reductions, ranging from 19.3% to a remarkable 26.1%, when compared against the strongest ABC baselines. This work validates that a data-driven search strategy can effectively navigate the vast and complex recipe space to uncover non-intuitive and highly effective optimization sequences, paving the way for more automated and targeted circuit design.

4.2 Future Work

While our framework has proven effective, there are several exciting avenues for future research and development that could build upon this foundation.

- **Sequence-Aware Recipe Encoding:** The current system’s primary limitation is its proxy-based scoring, which maps new recipes to the closest known recipe. The next logical step is to replace this with a true sequence model, such as a Recurrent Neural Network (RNN), LSTM, or Transformer. This would allow the predictor to understand the syntactic and semantic relationships between commands in a sequence, leading to more accurate predictions and a more finely-guided search.

- **Multi-Objective Optimization:** This project focused exclusively on power. A powerful extension would be to incorporate area (number of nodes) and delay (logic depth) into the cost function. The search algorithm could then be adapted to find a set of recipes representing the optimal Pareto front, allowing a designer to choose the best trade-off between Power, Performance, and Area (PPA) for their specific application.
- **Comprehensive Evaluation and Scaling:** The SA search should be run on the full suite of 25 benchmark designs to gather broader statistical evidence of its performance. Furthermore, exploring a larger search space by increasing the number of SA steps and tuning hyperparameters (e.g., temperature schedule, mutation rates) could yield even better results.
- **Advanced Transfer Learning:** Investigating the model’s ability to transfer knowledge across different circuit families is a key research direction. Techniques like domain adaptation or meta-learning could be explored to train a more general-purpose power predictor that adapts quickly to entirely new and unseen design architectures with minimal fine-tuning.

By pursuing these extensions, the framework can evolve into a more robust, versatile, and powerful tool for automated EDA optimization.